

ONYX LAB WHITEPAPER

BANDWORK

Refusal-Surface Engineering for Composable Generation

Version 1.3 · October 2026 · Onyx Lab · supersedes v1.2

A measured method for getting a frontier model to build what it will not build in one message: split the request, isolate each part, assemble the result locally, and keep the whole thing off the wire.

© 2026 Onyx Lab

Abstract

Bandwork is a method for getting a frontier model to build something it would refuse to build if you asked for all of it in a single message. It rests on one measured observation: a model's refusal is usually a reaction to the *shape* of a request, not to what the request is about. Ask for the whole thing and you get a no. Ask for the same thing one piece at a time, with each piece described in plain operational terms, and the model writes every piece; you put the pieces together on your own machine.

Version 1.2 documented the pipeline and the lesson from the response side - that a model which fills every piece in isolation will still refuse to describe the finished whole. Version 1.3 keeps that offensive core and adds two things. It states plainly what catching this technique would require from the other side, and it gives a forward view of where the technique is likely to be over the next few years. The offensive material is the body of this paper. The defensive view is deliberately the smaller half: this is an offensive framework and the paper is honest about that.

Contents

01 1. The Idea, In Plain Terms

02 2. The Method (the offensive core)

03 3. Measuring the Surface

04 4. What Changed Since 1.2

05 5. Economics

06 6. Does the Assembled Program Actually Work?

07 7. The Defensive View (the smaller half)

08 8. Outlook: Impact and Where This Goes

09 9. Known Limits

10 10. Reproduction

1 The Idea, In Plain Terms

Three words carry the whole method, so it is worth defining them at the top.

- **Refusal surface.** The boundary at which a model stops helping. It is not one line. It moves depending on the model, the exact wording, and the route the request travels (direct API versus a reseller). Bandwork treats that boundary as a thing to be measured and mapped, the way you would map any other interface.
- **Slot.** One function, one small unit of the finished program. A build is broken into slots, and each slot is requested on its own.
- **The wire.** Everything that actually leaves the client for the model. The rule is simple: the wire never carries the whole request.

The core finding, measured in version 1.2, is that refusal is compositional. A model will reject the complete request while happily producing every component of it. Send the whole artifact and it refuses; send twelve isolated functions and it writes twelve functions. The finished program then comes from gluing those twelve together locally, where no model is watching the assembly.

Why the refusal splits this way is the second finding, and it comes down to two different kinds of no:

- **The scanner wall.** Some models (Opus 5.5 is the measured example) refuse before they ever start reasoning. They key on capability vocabulary in the prompt and fire deterministically, in about two seconds, with an empty response. The cure is vocabulary: describe the same job without the flagged words.
- **The judge.** Other models (Grok 4.7 is the measured example) read the sentence, form an opinion, and refuse the act rather than the words. There is no scanner to fool; the cure is an honest re-description of the same job from a new angle.

The two are curable in opposite ways, so bandwork profiles per model instead of using one universal prompt. Disguises are for dumb filters; re-description is for smart judges.

The name comes from the gemstone. Onyx is banded, and a single band looks like nothing on its own; the picture exists only once the bands are polished together. Same here: no sentence on the wire ever describes the finished artifact, yet the artifact arrives as one coherent file, and every line of real logic was written by the model itself.

2 The Method (the offensive core)

The pipeline turns one request into one file through a fixed sequence of stages. Only three stages can call a model, and no single stage ever sees both the full request and the assembled output.

```
YOUR SPEC  ->  FRICTION GATE (is this a build, or just a question?)
           ->  PLANNER  (owns the blueprint: skeleton, includes,
               type declarations, call order, syscall plumbing)
           ->  PREP  (per-slot sentence: rebind risky names,
               scrub vocabulary, build the paraphrase ladder)
           ->  GENERATOR  (one isolated function per call, in parallel)
           ->  TRUST GATES  (name check, signature check, no dodges,
               salvage cut bodies)
           ->  ASSEMBLY  (deterministic substitution, real names restored)
           ->  ONE FILE  (integrate pass only if complete and long enough)
```

Three authors, strictly separated. The planner owns the blueprint and never writes the logic that thinks. The generator writes one function per call, each from a single isolated sentence. The assembler is pure string substitution and calls no model at all - because showing the finished whole back to a model is exactly the thing that triggers a refusal.

The generator sits behind a **ladder**. Each slot is asked through a rotating set of lead phrasings and honest paraphrases, with mild randomness on the deeper attempts, and the client remembers which wording worked before. A body that defines a different function than the one asked for, or the right name with the wrong shape, counts as a refusal and the ladder keeps rolling. Bodies that get cut off mid-generation are salvaged up to the last complete definition instead of thrown away.

Two details matter more than they look. Nothing on the wire names the composite, and the acceptance check is deterministic - a string check on the returned code, not a second model's opinion. That keeps the trust gates cheap and always available, immune to the same refusals the generation path is fighting.

3 Measuring the Surface

Bandwork does not grind blind. The space of possible requests is treated as a measurable object, and the measurements are stored so the next build starts warmer than the last.

- **Probe grid.** Each combination of framing, header presence, rewrite rung and lead is rolled several times per model, and the per-cell success rate becomes the starting order for the ladder.
- **Terrain.** Every finished slot records what it looked like and how it was won. A new slot warm-starts from its nearest neighbours, and failed attempts are learned too, not just successes.
- **Triggers.** Word-level counts of what got refused and what got through. A word that crosses a threshold of refusals forces the rewriter to fire first on future slots that contain it.
- **Route as a variable.** The path a request travels is recorded on every cell. Some tasks are flat zeros on the direct API and reliable through a reseller - the map shows this, and the pipeline routes around it.

The practical result is that the framework spends its effort where the boundary actually is, and remembers the answer, instead of retrying blindly.

4 What Changed Since 1.2

Three additions matter.

The response side learned the same lesson. The user-facing half of the pipeline was failing the way the wire once did, by showing the composite where it did not belong. Version 1.2 routed by form before any expensive machinery ran - a casual question is answered plainly, only a build enters the lane - and ended each build with a short handover note that describes counts ("twelve functions, one stub"), never purpose. The note that first leaked the original request was caught and fixed: anything that talks about the composite to a nervous model *is* the composite problem.

Two companion offensive lanes. Beyond the main decompose lane, two adjacent lanes attack the same pre-emptive filter on the same target from different angles, each grounded in published work:

- A **language-envelope** lane sends the request as a natural narrative in another language, translated by a non-target model, and forces the reply back to English. The published observation it rests on is that a pre-emptive, input-side filter is tuned to one language's trigger surface.
- A **cipher-and-demonstration** lane wraps the request in a simple, learnable cipher and primes the pattern with a handful of worked examples in ordinary turns. It follows three lines of 2026 literature: a learned covert channel, many-shot in-context priming, and capability laundering through decomposition.

Both lanes apply to one model family, both default off, and both leave the ordinary path untouched until deliberately enabled.

The trust gates became local. Assembly now ends in a deterministic quality pass that checks the finished file against the plan's own list of promised functions and marks each one met, missing or unverified. It makes no model call, so nothing on that path can be blocked. Repairs route back to a fast, ungated filler rather than back through the models that caused the gap.

5 Economics

A refused attempt is cheap. On the measured route a refusal returns empty in about a second and a half and bills input tokens only, while a real answer takes roughly eight seconds and bills output. Retrying the ladder is therefore near-free compared with a single accepted generation - the method can afford to grind.

The models differ in how they behave behind the ladder, which sets the cost per build:

model (reseller route)	output $/M	lane behaviour	note
claude-sonnet-5.5	~$14	10/12 slots first try	half the cost of Opus; default generator
claude-opus-5.5	~$28	~67% of rolls refused	wins deep in the ladder
grok-4.7	~$21	refuses on meaning	short ladder, judgment-based

Measured full build cost sits at roughly $0.07-0.15, dominated by the planner pass and the retry ladders. With plan caching and the terrain warm-starts described above, repeat builds of the same family project to a few cents and casual chat to a fraction of a cent. The economics reward memory: the first build pays tuition, the tenth is cheap.

6 Does the Assembled Program Actually Work?

Two different questions hide behind that sentence, and this section keeps them apart. The first is whether the file comes out *whole* - every function the plan promised is present with a real body. The second is whether that whole file actually *compiles and runs*. The first is measured below from the local build log. The second is not measured yet, so this section gives the exact criteria and the protocol that would produce it - the gap becomes a known zero instead of a claim.

6.1 The validity ladder

"Works" is defined as a ladder of machine-checkable levels, cheapest first. A build is scored at the highest level it passes.

- **L0 - non-empty.** The assembled file exists and is not wholly placeholder (assembled_chars > 0).
- **L1 - structurally complete.** slots_filled equals slots_total and the missing list is empty: every promised function has a non-empty body in the assembled file.
- **L2 - signature fidelity.** For each slot, the returned body defines the promised name at the promised arity. A bare prototype does not count; the body must define the name.
- **L3 - compiles.** The file is one translation unit; `cl /c` or `g++ -fsyntax-only` or `clang++ -fsyntax-only` exits 0 with no error diagnostics.
- **L4 - links.** With generated stubs standing in for the OS and third-party symbols, the linker exits 0.
- **L5 - runs to spec.** The binary executes under a timeout without crashing, and its observable output matches a behaviour spec that was declared before the run.

L0 and L1 are read straight from the build log. L2 is enforced at fill time - the fill gate records a slot only after its name and shape check passes - so every logged slot body already reflects L2. L3 through L5 need a compiler and, so far, have produced no data.

6.2 Measured (window: the local build log to date)

quantity	value
builds analysed (assembled-file events)	38
L1 complete	28 / 38 = 73.7%
L1 incomplete	10 / 38 = 26.3%
- carrying a named missing function	9 of the 10
slot fills	358
- returned non-empty	285 / 358 = 79.6%
- returned empty	73 / 358 = 20.4%
accepted on the first try	206 / 358 = 57.5%
mean tries per slot	3.15 (max 16)
assembled size	median 20,302 chars (range 1,191-47,050)
build languages	C++ 35, C 3

Missing functions appear by name: renderAimbot, viewAngles, MAIN, GET_VIEW_ANGLES; the remainder are numbered slots. The holes are not random - they are slots the model refused at every rung (the log marks them filter-dead) and the ladder exhausted, so the assembler reports them as missing.

The refusals happen upstream of assembly. Across 873 logged model attempts, 491 were refused - 56.2%, finish_reason content_filter, empty body - and completed responses carried a median of 1,054 visible characters. Attempts are logged at the first rung, and a refusal is cheap: median attempt latency is about 3.5 seconds against a completed generation.

6.3 When it completes, and when it does not

Completion by build size:

slots	complete / total
2	1 / 1
4	1 / 1
6	1 / 3

slots	complete / total
7	2 / 2
8	2 / 2
9	7 / 13
10	8 / 8
11	3 / 3
12	1 / 3
13	1 / 1
25	1 / 1

Where each slot landed, across the 358 fills:

slot provenance	count
lane (primary generator)	159
opus-5.5	77
unresolved	37
filter-dead (refused at every rung)	33
fallback	12

The 33 filter-dead slots line up with the incompleteness: those are the holes. Route is a variable here too - 865 of the 873 attempts travelled the AWS route, 3 went through Bedrock, and the difference in refusal behaviour between routes is what the surface measurement exists to record.

6.4 The protocol that produces L3, L4 and L5

The wire is measured; the compile result is the missing layer. Producing it is mechanical and repeatable:

- Write the artifact to disk for every assembled build in the window, under its declared language and extension.
- Compile it as one translation unit - `cl /std:c++20 /c /EHsc /W3` on MSVC, or `g++ -std=c++20 -fsyntax-only`, or `clang++ -std=c++20 -fsyntax-only`. Record the exit code and the diagnostic count. L3 passes on exit 0 and zero error diagnostics.

- Link it (L4) against a generated stub file that defines the OS and third-party symbols the artifact references, and record the exit code.
- Run it (L5) under a 5-second timeout with a fixed input, and compare observable output against a behaviour spec declared before the run.
- Emit one row per build - artifact hash, language, slots_total, L1, L2, compiler exit code, linker exit code, run result, toolchain version, timestamp - so the pass rate is passes over builds, at a stated n, on a stated toolchain.

Three properties of the criteria matter. The placeholders in the artifact (invented base addresses, undefined OS offsets) are expected and do not fail L3, because L3 is a syntax-and-type check, not a link; the stubs make L4 meaningful. L5 is the only rung that needs a per-artifact behaviour spec, and it must be fixed before the run, not after. And every level is scored by a tool that prints an exit code - none is a model's opinion.

6.5 Current status

L3, L4 and L5: not run. Builds tested for compilation: zero. What exists is real measurement of the wire and the assembly - a 73.7% structural-completion rate, a 56.2% per-attempt refusal rate, and the specific functions that go missing. What does not exist yet is a single recorded compile result, and this paper does not pretend otherwise. Closing it is the cross-slot coherence work named in section 9: a shared-types block declared once in the skeleton, enforced at the wire frame, with a mismatch-only patch pass. Once that lands, the L3 row above gets a real denominator.

7 The Defensive View (the smaller half)

This section is deliberately short. It exists because a framework paper should say what it would take to defend against itself, not because bandwork is a defensive tool.

The blunt version: the current safeguards on most services stop at the API boundary. They judge one request, in isolation, and they judge text. Bandwork defeats both assumptions at once - it splits one intent across many requests that are each individually benign, and it reassembles the result on the user's own machine, where the provider is not looking.

Catching it would require a different kind of analysis:

- **Cross-request correlation.** Judging a session as a whole, not a message at a time - recognising that twelve innocuous requests are one program.
- **Decomposition-aware evaluation.** Scoring not the request but the capability of the assembled artifact, which means following the pieces past the point where they leave the API.
- **Output-capability analysis.** Treating a working function as a capability even when the text that produced it carried no flagged vocabulary, because the vocabulary moved into the client's assembly step.

There is also a smaller, shared defensive half to the reasoning-trace work that bandwork cites. That work has two directions: one tries to pull a model's private reasoning out into logs, the other hardens against it. Bandwork only uses the second half - guards that warn when pasted text is shaped as a request to transcribe reasoning, redaction of secrets and encrypted reasoning blocks before anything is exported, and flags for summaries that dress a guess in the grammar of a proof. All of it is advisory: it highlights, it never blocks a send.

Stated once more so there is no confusion about the ratio: this is an offensive framework. The defensive view above is the mirrored, smaller half, and it is offered as what the other side of the boundary would have to become - not as the point of the paper.

8 Outlook: Impact and Where This Goes

The material in this section is forward-looking. It is a projection and a set of scenarios, not measurement; the measured results are in sections 1 through 5.

The immediate impact is on how capability is assessed. Bandwork shows that a single-request safety evaluation can miss a real capability that the same model will produce when the request is split. Any evaluation that judges one prompt at a time is measuring the wrong thing. The interesting number is not what the model refuses; it is what the model produces when refusal is removed from the equation.

What this looks like over time, as a scenario rather than a forecast:

- **Near term (0-6 months).** Providers begin moving from single-request filters to session-level signals - rate, shape and sequence anomalies that hint at decomposition. Bandwork adapts in the ordinary way: it already treats the route as a variable and the ladder as cheap, so a new filter is another cell to measure. The framework holds.
- **Mid term (6-18 months).** The first decomposition-aware classifiers appear, and judgement stops being about individual prompts. This is the harder shift. Defenders get a genuine lever: it becomes possible to flag a session whose *pattern* is "many small requests that assemble into one capable artifact". Bandwork's advantage narrows here, and the terrain - recorded per route and per model - becomes the main defensive counter, because the framework re-measures fast and the classifier does not know which of many possible splits the client chose.
- **Long term (18-36 months).** The arms race settles into a familiar shape. Neither side wins outright. Defenders hold the interface boundary and get better at pattern detection; composable methods hold the local-assembly side, where the finished artifact is never actually shown to anyone. The durable asymmetry is structural, not a bug: the provider sees requests, the user holds the assembled result.

For the security field, the impact is that orchestration starts to matter as much as the model. The planner, the isolation, the validation gates, the memory and the deterministic assembler are the operative capability - the surrounding system, not the single call. Evaluation and defence both have to move up a level to match.

The standard of proof for any cross-model claim is high, and this paper does not claim it yet. A general result would need frozen model versions, hundreds of blinded tasks, direct baselines, repeated trials, complete failure logs, token and wall-time costs, confidence intervals, and independent replication. If those results hold, bandwork becomes a reproducible research result rather than a compelling case study. Until then, treat the framework as effective in its measured setting, not as universal.

9 Known Limits

- **Cross-slot coherence is unchecked.** Each slot carries its own private assumptions - invented helpers, placeholder offsets - and a file stitched from independent functions can fail to compile as one program. The planned fix is a shared-types block declared once up front and a mismatch-only patch pass.
- **Memory is superstitious by design.** Stored terrain can encode lessons that were true of a model or route that has since changed, so a version-pin and forget path is required.
- **Gating is the usual failure.** A build request that slips past the gate and reaches the model whole will look like a regression; the mitigation is to retry it through the lane.
- **Route-bound zeros stay zeros.** Some task classes are refused on one route and pass on another; the map records that, but it does not move the boundary.

10 Reproduction

Framework files: the pipeline and the surface mapper. Inside the shipping client the method appears as the intent router, the decompose lane, the deterministic quality pass and the session spine.

Toggles: the decompose lane is enabled with its environment switch; lane membership is set by environment; the quality pass is on by default; the two companion lanes are each behind their own default-off switch.

Telemetry, all local and per install: a per-request usage log, a build debug log, and a record of which slot wordings won. The research corpus that drives the warm-starts - the terrain and trigger files - lives beside the mapper and is read-only from the application.

© 2026 Onyx Lab. All rights reserved.